시스템 프로그래밍 과제#2 1 차

22300650 전도원, 22300404 송준선

이름	학번	역할
전도원	22300650	과제 1-1, 1-2 작성, 보고서 작성
송준선	22300404	과제 1-3, 데모 캡쳐, make 작성

1. 소스코드 파일별 함수

파일	포함된 함수	설명
bmp.h	없음	BMP 헤더 구조체 정의와 함수 선언
bmp.c	read_header	공통 헤더 읽기
header.c	print_header	과제 1-1 헤더 출력
hexdump.c	hex_dump	과제 1-2 body 16 진수 출력
grayscale.c	make_gray	과제 1-3 grayscale 변환
main.c	print_usage, main	옵션을 확인하고 알맞은 기능 실행

2. 개별 함수의 기능

int read_header(FILE* fp, BITMAPFILEHEADER* fh, BITMAPINFOHEADER* ih)

fp 에서 파일 헤더 14 바이트와 정보 헤더 40 바이트를 fread 로 읽어 fh 와 ih 에 저장한다. bfType 이 BM 을 뜻하는 0x4D42 인지 확인해서 BMP 파일이 맞으면 0, 읽기에 실패하거나 BMP 파일이 아니면 -1 을 반환한다.

int print_header(char* filename)

과제 1-1. 파일을 rb 모드로 열고 read_header 로 헤더를 읽은 뒤, 두 헤더의 모든 멤버를 이름과 함께 한 줄씩 출력한다. 파일을 열 수 없거나 BMP 파일이 아니면 오류 메시지를 출력하고 -1 을 반환한다.

int hex_dump(char* in_name, char* out_name)

과제 1-2. 헤더를 읽은 뒤 fseek 으로 bfOffBits 위치로 이동하고, 파일 끝까지 fgetc 로 한 바이트씩 읽어 두 자리 16 진수로 출력 파일에 쓴다. 지금까지 출력한 바이트 수를 상대 주소로 사용해 8 바이트마다 줄을 바꾸고 줄 앞에 주소를 붙인다.

int make_gray(char* in_name, char* out_name)

과제 1-3. 24 비트와 32 비트 BMP 만 처리한다. 파일 전체를 malloc 으로 할당한 메모리에 읽고, 이중 for 문으로 픽셀마다 밝기 0.299 R + 0.587 G + 0.114 B 를 정수 계산으로 구해

B, G, R 에 같은 값을 넣는다. 한 행의 크기를 4 바이트 배수로 계산해 패딩을 건너뛰고, 결과를 wb 모드로 저장한다. 헤더는 그대로 유지된다.

void print_usage(char* name)

세 가지 옵션의 사용법을 화면에 출력한다.

int main(int argc, char* argv[])

첫 번째 인자와 인자 개수를 확인해 -h 이면 print_header, -o 이면 hex_dump, -g 이면 make_gray 를 호출한다. 맞는 경우가 없으면 print_usage 를 호출한다. 성공하면 0, 실패하면 1 을 반환한다.

3. 자료구조

BMP 파일은 BITMAPFILEHEADER 14 바이트, BITMAPINFOHEADER 40 바이트, 픽셀 데이터 순서로 저장된다. 픽셀 데이터는 bfOffBits 위치에서 시작한다.

픽셀은 B, G, R 순서로 저장되고 32 비트 이미지는 그 뒤에 알파 값 A 가 붙는다. 한 행의 크기는 4 바이트의 배수가 되도록 뒤에 0 이 채워진다.

3-1. BITMAPFILEHEADER, 14 바이트

멤버	타입	의미
bfType	unsigned short	파일 종류, BM
bfSize	unsigned int	파일 전체 크기
bfReserved1, bfReserved2	unsigned short	사용하지 않음, 항상 0
bfOffBits	unsigned int	픽셀 데이터 시작 위치

3-2. BITMAPINFOHEADER, 40 바이트

멤버	타입	의미
biSize	unsigned int	정보 헤더 크기, 40
biWidth, biHeight	int	가로, 세로 픽셀 수
biPlanes	unsigned short	항상 1
biBitCount	unsigned short	픽셀 하나의 비트 수, 24 또는 32 등
biCompression	unsigned int	압축 방식, 0 이면 압축 안 함
biSizeImage	unsigned int	픽셀 데이터 크기
biXPelsPerMeter, biYPelsPerMeter	int	가로, 세로 해상도
biClrUsed, biClrImportant	unsigned int	사용한 색상 수, 중요한 색상 수

컴파일러가 구조체 멤버 사이에 빈 공간을 넣으면 BITMAPFILEHEADER 가 16 바이트가 되어 fread 로 읽은 값이 밀린다. 이를 막기 위해 pragma pack 1 지시어로 구조체를 1 바이트 단위로 정렬했다.

4. 프로그램 사용법

1. 컴파일: make
2. 과제 1-1 헤더 출력: ./hw2BMP -h khj.bmp
3. 과제 1-2 body 를 16 진수 텍스트로 저장: ./hw2BMP -o khj.bmp body.txt
4. 과제 1-3 grayscale 변환: ./hw2BMP -g khj.bmp khj_gray.bmp
5. 생성된 파일 정리: make clean

5. 데모 화면

컴파일

make — 빌드

```
$ make
gcc -Wall -c main.c
gcc -Wall -c bmp.c
gcc -Wall -c header.c
gcc -Wall -c hexdump.c
gcc -Wall -c grayscale.c
gcc -Wall -o hw2BMP main.o bmp.o header.o hexdump.o grayscale.o
```

과제 1-1 헤더 출력

1-1) hw2BMP -h

```
$ ./hw2BMP -h khj.bmp
===== BITMAP FILE HEADER =====
bfType          : BM
bfSize          : 18514
bfReserved1     : 0
bfReserved2     : 0
bfOffBits       : 54

===== BITMAP INFO HEADER =====
biSize          : 40
biWidth         : 71
biHeight        : 65
biPlanes        : 1
biBitCount      : 32
biCompression   : 0
biSizeImage     : 0
biXPelsPerMeter : 0
biYPelsPerMeter : 0
biClrUsed       : 0
biClrImportant  : 0
```

과제 1-2 body 16 진수 출력

1-2) hw2BMP -o

```
$ ./hw2BMP -o khj.bmp body.txt
BMP body 18460 바이트를 body.txt 에 저장했습니다.
$ head -n 8 body.txt
00000000: 00 01 02 FF 00 00 01 FF
00000008: 00 00 00 FF 03 03 03 FF
00000010: 03 01 01 FF 04 02 02 FF
00000018: C1 BF BF FF 96 94 94 FF
00000020: 04 00 01 FF 52 4F 51 FF
00000028: DE DD E1 FF 3E 3E 44 FF
00000030: 88 8B 90 FF 19 1C 24 FF
00000038: 00 01 09 FF 14 1C 29 FF
...
$ tail -n 3 body.txt
00004808: A1 99 9A FF A2 9A 9B FF
00004810: A2 9A 9B FF A3 9B 9C FF
00004818: A3 9B 9C FF
$ wc -l body.txt
    2308 body.txt
```

과제 1-3 grayscale 변환

1-3) hw2BMP -g

```
$ ./hw2BMP -g khj.bmp khj_gray.bmp
grayscale 이미지를 khj_gray.bmp 에 저장했습니다.
$ file khj_gray.bmp
khj_gray.bmp: PC bitmap, Windows 3.x format, 71 x 65 x 32, cbSize 18514, bits offset 54
```

원본 khj.bmp



변환 khj_gray.bmp

